1. System Purpose

The Predictive Execution Cockpit provides a real-time operational view of an execution engine. It visualizes:

Regime state

Risk / Impact / Slippage / Latency

Arbitration & federation decisions

Anomaly detection

Event timelines

Quadrant classification

Sync drift

Heartbeat

The system is stateless, scalable, model-agnostic, and data-agnostic.

2. High-Level Architecture

Components:

Backend API (FastAPI)

Provides all metrics and decision logic via JSON endpoints.

Frontend Cockpit (Static HTML/JS)

Polls backend every second and renders cockpit panels.

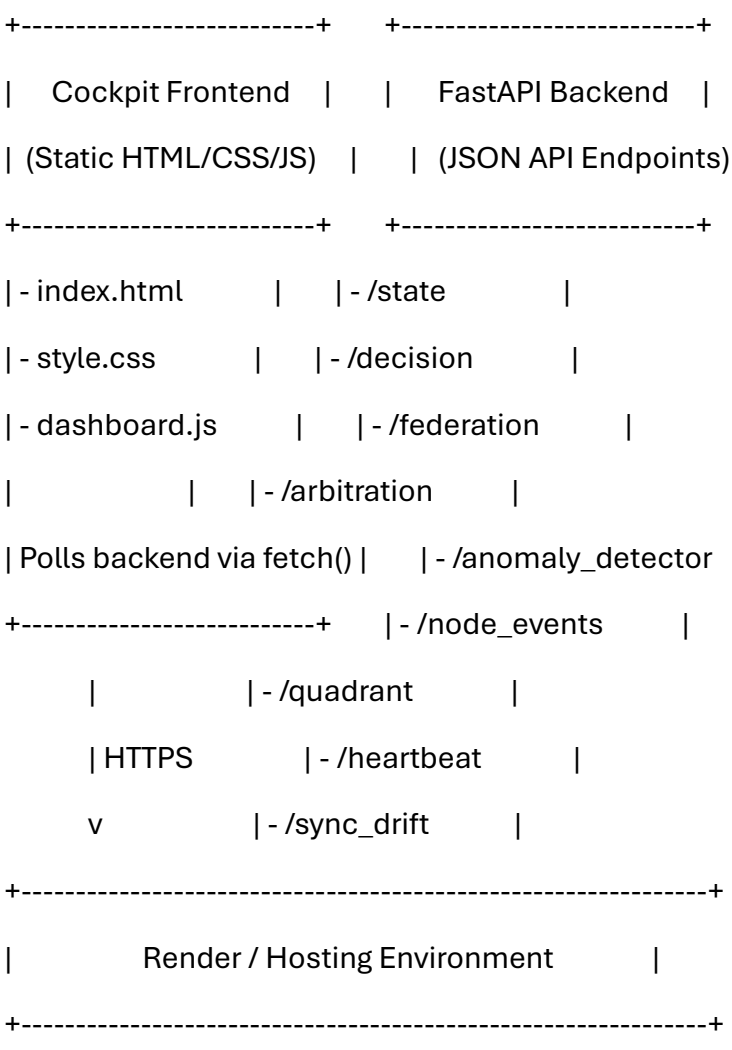
Deployment

Backend = Render Web Service

Frontend = Render Static Site

Communication = HTTPS + CORS

Architecture Diagram (ASCII):



3. Backend Architecture

Tech stack:

Python 3

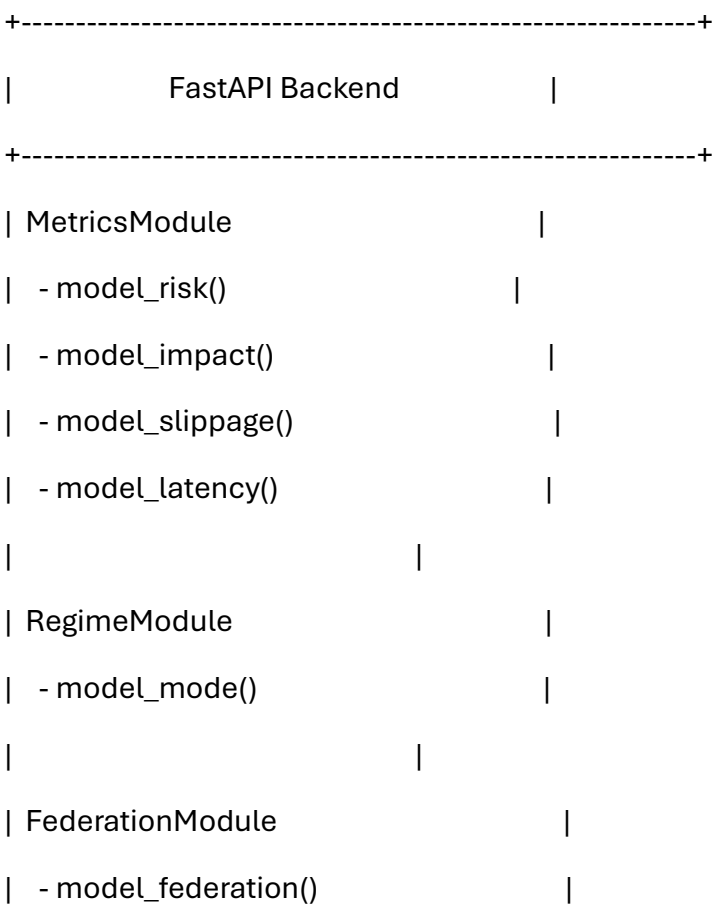
FastAPI

Gunicorn + Uvicorn worker

Stateless

No database required

Backend Component Diagram (UML):



```
|                                  |
| ArbitrationModule                |
|   - arbitration()                |
|                                  |
| AnomalyModule                    |
|   - anomaly_detector()           |
|                                  |
| QuadrantModule                   |
|   - quadrant()                   |
|                                  |
| SyncModule                       |
|   - sync_drift()                 |
|                                  |
| HeartbeatModule                  |
|   - heartbeat()                  |
+----------------------------------------------------------+
```

4. Backend Data Flow

Frontend -> Backend: GET /state

Backend -> Metrics Module: compute risk, impact, slippage, latency

Metrics Module --> Backend: metrics

Backend -> Regime Module: compute mode

Regime Module --> Backend: mode

Backend --> Frontend: JSON {risk, impact, slippage, latency, mode}

Frontend -> Cockpit Panels: update display

5. Backend Sequence Diagram

Frontend -> Backend: GET /state

Backend -> MetricsModule: compute metrics

MetricsModule -> Backend: return metrics

Backend -> RegimeModule: compute mode

RegimeModule -> Backend: return mode

Backend -> Frontend: JSON response

Frontend -> DOM: update cockpit panel

6. Backend Logic Specification

6.1 Risk

risk = clamp(0, 1, 0.35 + sin(t/8)*0.25 + noise)

6.2 Impact

impact = clamp(0, 1, 0.40 + cos(t/10)*0.20 + noise)

6.3 Slippage

slippage = clamp(0, 1, 0.30 + sin(t/6)*0.30 + noise)

6.4 Latency

latency = max(1, 12 + sin(t/4)*6 + noise)

6.5 Regime Mode

score = (risk + impact + slippage) / 3

if score < 0.33: OK

elif score < 0.66: WARN

else: CRITICAL

6.6 Federation

Outputs: risk, impact, slippage

Selects: model1 / model2 / model3

6.7 Arbitration

Uses mode

Returns final_mode

6.8 Anomaly Detection

risk_spike = risk > 0.85

impact_jump = impact > 0.85

slippage_jump = slippage > 0.85

latency_spike = latency > 20

6.9 Quadrant

LOW: risk<0.33 AND impact<0.33

MEDIUM: risk<0.66 AND impact<0.66

HIGH: risk<0.66 AND impact>=0.66

CRITICAL: else

6.10 Sync Drift

drift_ms = random(0..250)

status = "OK" if drift_ms < 120 else "DRIFTING"

6.11 Heartbeat

alive = true

timestamp = time.now()

7. Backend API Specification

Endpoint	Purpose	Output
/	Health	{status:"ok"}
/state	Core regime state	risk, impact, slippage, latency, events
/decision	Automation decision	main_mode, finalMode
/federation	Model federation	outputs, federation
/arbitration	Final arbitration	final_mode
/anomaly_detector	Anomaly flags	risk_spike, latency_spike
/node_events	Recent events	list of events
/quadrant	Risk/impact quadrant	quadrant
/heartbeat	Liveness	alive
/sync_drift	Clock drift	drift_ms

8. Frontend Cockpit Design

Tech stack

- HTML
- CSS
- JavaScript
- No frameworks

Layout

Grid of 9 panels:

- STATE
- DECISION
- FEDERATION
- ARBITRATION
- ANOMALY
- EVENTS

- QUADRANT
- HEARTBEAT
- SYNC DRIFT

Frontend Component Diagram



Frontend Sequence Diagram

User -> Browser: load index.html

Browser -> dashboard.js: execute

dashboard.js -> Backend: GET /state

Backend -> dashboard.js: JSON

dashboard.js -> DOM: update panel

Repeat every 1 second

9. Frontend Data Flow

const BASE_URL = "https://predictive-execution-1.onrender.com";

fetch(BASE_URL + "/state")

fetch(BASE_URL + "/decision")

fetch(BASE_URL + "/federation")

fetch(BASE_URL + "/arbitration")

fetch(BASE_URL + "/anomaly_detector")

fetch(BASE_URL + "/node_events")

fetch(BASE_URL + "/quadrant")

fetch(BASE_URL + "/heartbeat")

fetch(BASE_URL + "/sync_drift")

10. Deployment Design

Backend (Render Web Service)

Build Command:

pip install -r requirements.txt

Start Command:

gunicorn server:app --worker-class uvicorn.workers.UvicornWorker

Files:

- **server.py**
- **requirements.txt**

Frontend (Render Static Site)

Publish Directory:

- **. or frontend/**

Files:

- **index.html**
- **style.css**
- **dashboard.js**

Replace simulated metrics with real feeds

- **Risk from exposure**

- **Impact from footprint**
- **Slippage from benchmarks**
- **Latency from telemetry**

Real federation

- **Multi-model scoring**
- **Confidence weighting**
- **Reliability tracking**

Real arbitration

- **Rule-based overrides**
- **Safety constraints**
- **Multi-model arbitration**

Real anomaly detection

- **EWMA**
- **Z-score**
- **Regime-dependent thresholds**

Cockpit upgrade

- **Charts**
- **Heatmaps**
- **Timelines**
- **Alerts**
- **Regime indicators**